

# Updated RAG Ingestion Framework (2026)

To ingest ~35 MB of mixed-text data into a RAG system (as of 2026), we extend the original four-method pipeline <sup>1</sup> <sup>2</sup> with cutting-edge embedding models, agentic orchestration, and efficient storage. First, **preprocess and clean** all source files (e.g. convert PDFs to clean text/Markdown, remove headers/footers) as a “clean-first” step <sup>3</sup>. Metadata (source, section, date) should be tagged on each chunk to improve retrieval accuracy <sup>4</sup>. Then split text via four complementary chunkers: fixed-size, sentence-based, semantic, and recursive-hierarchical. Finally, embed each chunk and index it in a vector store for retrieval. We now outline each stage with 2026 updates:

## 1. Preprocessing & Cleaning

- **Convert and Sanitize:** Extract text using robust parsers or OCR, especially for scans. As noted, PDFs often introduce noise (misplaced headers/footers) <sup>5</sup>. Remove boilerplate before chunking (“do not try to chunk a PDF; get to clean Markdown first” <sup>3</sup>). Convert to a clean text/Markdown format (tables and images may need special handling).
- **Sectioning and Metadata:** Identify document structure (headings, subheadings) and assign metadata (source, section, date) to each chunk <sup>4</sup>. These tags can be stored in the index to support relevance ranking (e.g. prefer newer dates) and filtering.
- **Overlap and QA:** When chunking (below), include overlaps (e.g. last sentence(s) of prior chunk) to preserve context <sup>6</sup>. After initial ingestion, sample and verify chunk quality (per the 10-step pipeline) <sup>7</sup>. Iterate on cleaning or splitting heuristics based on retrieval tests.

## 2. Chunking Strategies

We apply all four methods in parallel (asynchronously) to maximize retrievability <sup>2</sup> <sup>8</sup>. Each method produces overlapping chunks, which are embedded and indexed separately:

- **Fixed-Size Chunking:** Split text into uniform segments by token/character count (e.g. 1024–2048 tokens each) with overlap. This is fast and easy, but risks cutting mid-sentence <sup>9</sup> <sup>10</sup>. (We can do this programmatically without an LLM.) Good for ensuring a strict chunk size limit.
- **Sentence-Based Chunking:** Group whole sentences into chunks (e.g. 3–5 sentences per chunk) <sup>11</sup>. This preserves linguistic boundaries (no sentence is split) <sup>12</sup>. It can also be done algorithmically using an NLP sentence-splitter (spaCy, NLTK, etc.). Overlap is handled by including one or more trailing sentences from the previous chunk.
- **Semantic Chunking:** Break text by topic/coherence. We embed smaller units (e.g. paragraphs or sentences) and measure vector similarity: where similarity drops below a threshold, we mark a chunk boundary <sup>13</sup>. In practice, we could cluster embeddings of adjacent sentences or paragraphs, or use an LLM to detect topic shifts. This yields chunks that each cover a single theme <sup>8</sup> <sup>14</sup>. Semantic chunking often requires iterative embedding & clustering and is computationally heavier, but yields high-quality, topically coherent segments.
- **Recursive/Hierarchical Chunking:** Preserve document hierarchy (headings/subheadings) by splitting recursively. For example, treat each top-level section as a chunk, then subdivide into

subsections/paragraphs as needed <sup>15</sup> <sup>16</sup> . This is effective for well-structured Markdown or HTML documents: simply parse the headings and break at those points <sup>17</sup> . Each chunk stores its parent ID and heading text in metadata. This can be done with a parser rather than an LLM, given the files are mostly structured text.

- **Hybrid/Agentic Chunking:** In addition to the above, hybrid techniques (sliding windows, multi-pass splits) can be used <sup>18</sup> . Notably, **agentic chunking** (using an LLM to choose split points) is now feasible <sup>19</sup> . We could have a lightweight agent that reads text and outputs semantic breaks (especially useful for very messy or code-mixed content). However, fixed/sentence/hierarchical chunking can mostly run without heavy models.

*Implementation note:* All four chunking passes can run concurrently across files (multi-threaded/asynchronously). Libraries like MemVid support parallel ingestion of segments <sup>20</sup> . We should ensure chunks overlap and include context (as shown above).

### 3. Embedding Models & Dimensions

**New 2026 embedding models:** The state-of-the-art text embedding models include **Qwen3-Embedding** (Alibaba) and **Gemini Embedding-001** (Google) <sup>21</sup> <sup>22</sup> . Key properties:

- **Qwen3-Embedding-8B / 4B:** Released late 2025, these open models support very long context (32,768 tokens) <sup>23</sup> <sup>24</sup> . The 8B model outputs 4096-dimensional embeddings; the 4B outputs 2048 dims <sup>22</sup> . They excel on benchmarks (second only to Gemini) <sup>25</sup> . Available via OpenRouter at ~\$0.01 (8B) or \$0.02 (4B) per million input tokens <sup>23</sup> <sup>24</sup> . They also support flexible (Matryoshka) output dims and quantization <sup>22</sup> .
- **Gemini Embedding-001:** Google's new top-ranked model (launched mid-2025). It supports up to 2048 input tokens and outputs 3072 dimensions by default, with MRL enabling output truncation to 1536 or 768 <sup>21</sup> . (Gemini's predecessor, text-embedding-004, is being deprecated in Jan 2026 <sup>26</sup> .) Gemini-001 costs ~\$0.15 per million tokens <sup>27</sup> , but has very high accuracy across domains.
- **Other Models:** For smaller-scale or cost-conscious needs, use compact embeddings. For example, **EmbeddingGemma-300M** (300M size) gives 768-dim outputs and runs in ~200MB RAM <sup>28</sup> . Jina's Embeddings V4 is also multimodal with code support (though license may restrict usage) <sup>29</sup> . For code specifically, note that Qwen3 models are trained on code and support 100+ programming languages <sup>30</sup> .

**Embedding chunk size:** In practice, we choose chunk lengths that balance semantic completeness and token limits. Gemini caps at 2048 tokens <sup>21</sup> , so for safety we might limit chunks to ~1–1.5K tokens for Gemini (with overlap). Qwen3 can handle much longer (up to 32K) <sup>22</sup> ; we might still use chunks of a few thousand tokens (1–3K) to keep retrieval focused and reduce embedding cost. Each chunk's text is embedded into its vector (dim up to 4096).

**Cost estimates:** Embedding the full 8.75M tokens of text would cost roughly **\$0.09 with Qwen3-8B vs \$1.31 with Gemini-001** (at \$0.01 vs \$0.15 per million) <sup>23</sup> <sup>27</sup> . For retrieval: if you issue ~1k queries/week (~1k tokens/query), that's 1M tokens/week, costing ~\$0.01/week on Qwen vs \$0.15/week on Gemini. Over a year, that's roughly \$0.50 vs \$7.80 <sup>23</sup> <sup>27</sup> . Thus Qwen is far cheaper for large-scale embedding, while Gemini provides leading accuracy if its higher cost is acceptable. (OpenRouter also offers some free or lower-cost providers for these models.)

## 4. Asynchronous Agentic Orchestration

To manage parallel chunking, embedding, and retrieval, we use an **agentic pipeline** (meta-controller with sub-agents) <sup>31</sup> <sup>32</sup>. Conceptually, one could define the following components as autonomous agents (e.g. via Anthropic’s Agentic SDK or similar frameworks):

- **Ingestion Agents:** For each chunking method (fixed, sentence, semantic, hierarchical), spin up workers that read files and output chunks. Fixed/sentence/hierarchical agents can run rule-based code; semantic chunking can involve an LLM or embedding calls to find topic breaks. MemVid’s multi-threaded ingest <sup>20</sup> can be used to parallelize at scale.
- **Embedding Agents:** Feed each new chunk to the embedding API (OpenRouter) and index the vector. These calls can be batched asynchronously. Using a shared vector store like MemVid (see below) allows write-concurrency across agents.
- **Indexing Agent:** A centralized process (or MemVid itself) which writes the embeddings and metadata into the memory store.
- **Query Analysis Agent:** At retrieval time, an initial agent examines the user’s query and determines which domain or RAG index to use (e.g. code vs writing vs research papers). It may decompose a complex query into sub-queries. This is the “query analysis” step of an agentic RAG <sup>33</sup>.
- **Retrieval Agents:** Multiple retrievers may be called in parallel: for example, a “semantic retriever” that uses vector similarity on the semantic-chunk index, and a “keyword retriever” that does a full-text search or hybrid search. The agent collects top-K hits from each and may deduplicate or re-rank (possibly using a reranker model). This multi-step retrieval is analogous to AutoGen/DSPy agentic designs <sup>31</sup>.
- **Response Agent:** Finally, one agent or LLM synthesizes the retrieved content into an answer, optionally applying few-shot or chain-of-thought reasoning on the aggregated chunks.

Crucially, we can **decouple responsibilities**: fixed/sentence chunking can be done with simple scripts (no LLM needed) and scales cheaply, while semantic/hierarchical chunking can leverage larger models for quality. Agentic frameworks (e.g. AutoGen) already support such multi-agent collaboration on retrieval tasks <sup>31</sup>. We can run these agents asynchronously or in parallel (as the workload allows) to speed up ingestion. For example, MemVid’s “parallel\_segments” feature <sup>20</sup> suggests the pipeline can ingest multiple segments concurrently.

## 5. Code vs. Text Processing

Since the project includes both natural text (writings, papers) and code, we treat them with slightly different strategies:

- **Code Embedding:** Code can be chunked by logical units (functions, classes) using syntax-aware parsers (e.g. Tree Sitter or the code chunker described by CintraAI <sup>34</sup>). This is algorithmic (no need to split mid-function). The resulting code chunks are still treated as text for embedding. Qwen3 embeddings are “multilingual” (including programming languages) <sup>30</sup>, so the same Qwen3 model can embed code snippets with good results. Alternatively, one could use specialized code embeddings (VoyageCode3, Jina code adapters, etc.) but Qwen3’s high performance often makes separate models unnecessary.
- **Metadata Tagging:** Store a flag in each chunk’s metadata indicating if it’s code or text. This allows the query agent to prefer code chunks when appropriate.

- **Separate Indices (Optional):** You may choose to maintain two vector indexes (e.g. two MemVid files) – one for prose and one for code – if that simplifies querying. Otherwise, storing all in one index with type tags can work, letting the retriever agent filter by type.

## 6. Memory Storage: MemVid

For storage, a modern approach is to use **MemVid** (a single-file memory) instead of a separate server-based vector DB <sup>35</sup>. MemVid packages all chunks, embeddings, and metadata into a compressed “video-like” file. Its frame-based design allows ultra-efficient compression (using video encoding techniques) and append-only writes <sup>36</sup>. In practice:

- **Single File Store:** All chunks can be appended to `knowledge.mv2` (for example). This file is portable and versioned.
- **Efficient Retrieval:** MemVid supports fast vector similarity search (with HNSW) and also full-text search if enabled <sup>37</sup>. Its “Smart Recall” achieves sub-5ms retrieval on local disk <sup>38</sup>. Because it compresses overlapping content well, large data sets (like 35 MB text) remain compact.
- **Parallel Ingestion:** MemVid allows multi-threaded ingestion <sup>20</sup>. We can feed it embeddings from all chunking agents in parallel.
- **Memory Timeline:** It behaves like a rewindable timeline (useful if we ever incrementally update the knowledge base) <sup>39</sup>.
- **No Ops Required:** Using MemVid means no vector DB server to maintain. It automatically handles indexing (including optional lexicon and vector indices).

In short, MemVid turns our RAG index into a compressed, self-contained artifact. For 2026, it represents an efficient alternative to traditional DBs, especially for personal or offline use.

## 7. Retrieval Workflow

With the data indexed, handling a user query might proceed as follows:

1. **Query Analysis:** An agent (LLM) classifies the query intent (e.g. “This is a coding question” vs “this is a research query” vs “general”) and possibly reformulates it. Based on this, it chooses which chunks to retrieve (e.g. search the code index vs text index) and what strategy (semantic vs keyword hybrid).
2. **Vector Search:** Use MemVid’s search to find top-K vectors similar to the query embedding (using the same embedding model, likely Qwen3 or Gemini). If multiple chunk sets exist (fixed/sentence/semantic), you may query all and merge results. Hybrid search (vector + BM25) can be enabled to mix keyword matching <sup>40</sup>.
3. **Ranking and Filtering:** Apply any additional reranking. For example, one could feed the top candidates into a Qwen3-Reranker or a mini LLM to score relevance. Use metadata tags (source, recency, type) to boost or filter results <sup>4</sup>.
4. **Multi-Step Retrieval:** Optionally, follow agentic RAG multi-step patterns <sup>31</sup>: one agent retrieves initial facts, another verifies or expands them, etc., before final answer generation.
5. **Answer Synthesis:** Finally, an LLM (e.g. Claude or GPT-4) is invoked with the retrieved snippets in context to generate the answer. This generation agent may also update its “memory” of the conversation (using RAG on previous dialogue).

## 8. Existing Frameworks and Tools

Several 2025–2026 tools overlap with this design:

- **RAGFlow (Infiniflow):** An open-source RAG engine that explicitly combines retrieval with agentic planning <sup>32</sup>. It provides a drag-drop pipeline interface (like UltraRAG) <sup>41</sup> and can orchestrate multi-step RAG.
- **LangChain / Haystack / Morpheus:** Common RAG frameworks. LangChain (code-based) provides building blocks for chunking and retrieval <sup>42</sup>. Haystack is enterprise-ready with PDF pipelines. Morpheus specializes in page-level multimodal RAG. All can be extended with custom components (e.g. to use Qwen3 embeddings).
- **Agentic RAG Libraries:** AutoGen (Microsoft) and DSPy (Meta's DeepSpeed) let you define multiple specialized agents (retriever, validator, summarizer) working in concert <sup>31</sup>. Although they target question answering, the principles apply to document ingestion.
- **Embedding/Vector Tools:** OpenRouter (for model APIs), and tools like RedisVector for hybrid search. MemVid itself is relatively new (as of 2025) but fits perfectly as our storage layer.

No single off-the-shelf system does *all* of this out-of-the-box; rather, the solution is to combine best-in-class components. For example, one could use RAGFlow or LangChain for orchestration, OpenRouter for embeddings, and MemVid for storage. The key is to configure each step as above.

## 9. Recommended Pipeline (Summary)

Based on the above, a 2026-optimized workflow would be:

- **Preprocess:** Clean and normalize all Markdown/text (remove noise).
- **Chunk:** In parallel, apply fixed-size, sentence, semantic, and heading-based splitting to each document. Add overlaps and metadata (source, headings, type). For code files, split by syntax (functions).
- **Embed:** Use Qwen3-Embedding (8B or 4B) to embed all chunks (cheap, high-quality). Optionally embed code with Qwen3 as well. Store vectors in MemVid.
- **Index:** In MemVid, enable both vector and full-text search, and activate features (e.g. `parallel_segments`) for performance <sup>20</sup>.
- **Query:** For each user query, run a short agentic workflow: classify query, retrieve via MemVid (semantic + hybrid search), possibly rerank with an embedding or LLM, then feed top chunks to the answer LLM.

This framework updates the original 2024 methodology with 2026 advances: the latest embedding models (Qwen3, Gemini) <sup>23</sup> <sup>21</sup>, asynchronous agentic chunking and retrieval <sup>16</sup> <sup>31</sup>, and efficient “video” storage of embeddings <sup>36</sup>. By following these steps, you build a scalable RAG solution that handles large text and code corpora with minimal manual intervention.

**Sources:** Concepts and best practices are drawn from the attached RAG meta-analysis <sup>2</sup> <sup>15</sup> and recent literature on chunking and embeddings <sup>10</sup> <sup>21</sup> <sup>36</sup>, as well as documentation of the latest models <sup>23</sup> <sup>22</sup>. These sources illustrate the described methods and justify the model choices and architecture above.

1 2 3 4 5 6 7 8 9 11 12 13 15 17 33 RAG Processing Framework Meta-Analysis and Implementation Guide.md

[file:///file\\_00000000896c722f891aa8ee06143efa](file:///file_00000000896c722f891aa8ee06143efa)

10 14 16 18 19 LLM Chunking: How to Improve Retrieval & Accuracy at Scale | Redis

<https://redis.io/blog/llm-chunking/>

20 35 36 37 38 39 GitHub - memvid/memvid: Memory layer for AI Agents. Replace complex RAG pipelines with a serverless, single-file memory layer. Give your agents instant retrieval and long-term memory.

<https://github.com/memvid/memvid>

21 26 27 Gemini Embedding now generally available in the Gemini API - Google Developers Blog

<https://developers.googleblog.com/gemini-embedding-available-gemini-api/>

22 Qwen/Qwen3-VL-Embedding-2B · Hugging Face

<https://huggingface.co/Qwen/Qwen3-VL-Embedding-2B>

23 Qwen3 Embedding 8B - API, Providers, Stats | OpenRouter

<https://openrouter.ai/qwen/qwen3-embedding-8b>

24 Qwen3 Embedding 4B - API, Providers, Stats | OpenRouter

<https://openrouter.ai/qwen/qwen3-embedding-4b>

25 Qwen3 Embedding

<https://simonwillison.net/2025/Jun/8/qwen3-embedding/>

28 29 30 The Best Open-Source Embedding Models in 2026

<https://www.bentoml.com/blog/a-guide-to-open-source-embedding-models>

31 40 41 42 2025 Ultimate Guide to Open-Source RAG Frameworks for Developers | Morphik Blog

<https://www.morphik.ai/blog/guide-to-oss-rag-frameworks-for-developers>

32 GitHub - infiniflow/ragflow: RAGFlow is a leading open-source Retrieval-Augmented Generation (RAG) engine that fuses cutting-edge RAG with Agent capabilities to create a superior context layer for LLMs

<https://github.com/infiniflow/ragflow>

34 Mastering Code Chunking for Retrieval Augmented Generation | by Joe Shamon | Medium

[https://medium.com/@joe\\_30979/mastering-code-chunking-for-retrieval-augmented-generation-66660397d0e0](https://medium.com/@joe_30979/mastering-code-chunking-for-retrieval-augmented-generation-66660397d0e0)